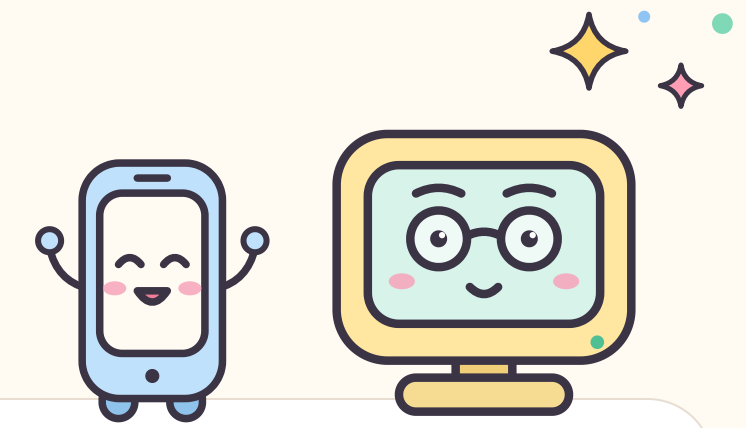




第 5 集 · 约 24 分钟

协议③：防重放与文件名

录下来再发一遍不行，起个坏名字也不行

配套阅读：书的第 2 章



从零写出 LanShare

这一集你会学到

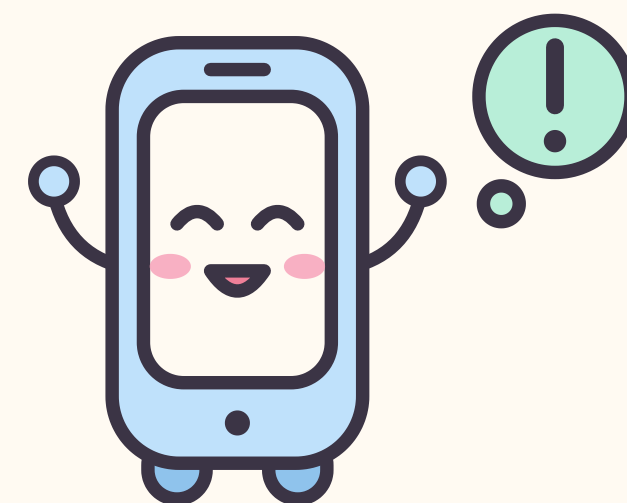
1. 防重放滑动窗口：允许乱序，拒绝重复和太旧的计数器
2. 先检查、解密成功再登记：别让伪造请求占掉合法计数器
3. 文件名清理：路径穿越、设备名、看不见的双向控制符
4. String 和字符串切片，按字节截断不能切在汉字中间
5. 写一个测速程序：examples 目录和 release 构建



先想一个问题

窃听者看不懂密文。可要是捣蛋鬼把一条“发文字”的请求录下来，原样再发一遍呢？

如果老电脑不记账，这条请求完全合法：密钥对、计数器对、印章也对，同一条文字会再出现一次。





为什么不能要求“计数器严格递增”

- 最直接的想法：只接受比上一个大的计数器
- 但浏览器是并发发请求的：轮询 + 3 个上传块同时在路上
- 滑动窗口：记住最高的计数器，以及它下面 4096 个里哪些用过



最高计数器是 10000 时

来了一个计数器	结果	为什么
0	拒绝	计数器从 1 开始
9998（没见过）	接受	在窗口里、没用过，乱序也没关系
9998（第二次）	拒绝	已经用过，这是重放
5904	拒绝	小于等于 10000 - 4096，比窗口还旧
5905	接受	窗口里最旧的一个，没用过
20000	接受	比最高的还大，窗口跟着往前滑



先检查，解密成功后再登记



- 如果 “一检查就登记”
- 捣蛋鬼用伪造的密文发一堆计数器，解密全失败，计数器却都被占掉了
- 所以拆成两步：check 只看不记，解密成功之后再 commit



replay.rs: 窗口的数据

replay.rs crates/lanshare-proto/src/

第 1-20 行 • 共 20 行

```
1  //! 防重放：滑动窗口。
2  //!
3  //! 浏览器会并发发请求（轮询 + 3 个上传块），到达顺序不一定等于计数器顺序，
4  //! 所以不能要求“严格递增”，而是记住最近 4096 个计数器里哪些已经用过。
5
6  use std::collections::BTreeSet;
7
8  /// 窗口大小：比最高计数器小 4096 及以上的请求一律拒绝。
9  pub const REPLAY_WINDOW: u64 = 4096;
10
11  #[derive(Debug, Default, Clone)]
12  pub struct ReplayWindow {
13      highest: u64,
14      seen: BTreeSet<u64>,
15  }
16
17  impl ReplayWindow {
18      pub fn new() -> Self {
19          Self::default()
20      }
```



check 和 commit

replay.rs
crates/lanshare-proto/src/
第 29-48 行 · 共 48 行

```

29         return false;
30     }
31     !self.seen.contains(&ctr)
32 }
33
34 /// 登记一个已经成功使用的计数器，并丢掉窗口外的旧记录。
35 pub fn commit(&mut self, ctr: u64) {
36     self.seen.insert(ctr);
37     if ctr > self.highest {
38         self.highest = ctr;
39         let floor = self.highest.saturating_sub(REPLAY_WINDOW);
40         self.seen = self.seen.split_off(&(floor + 1));
41     }
42 }
43
44 /// 当前记住的计数器个数（测试用，确认内存有界）。
45 pub fn tracked(&self) -> usize {
46     self.seen.len()
47 }
48 }

```



同样的窗口，用 Python 写

Python (照着写一个)

```
1 def commit(self, ctr):
2     self.seen.add(ctr)
3     if ctr > self.highest:
4         self.highest = ctr
5         floor = max(self.highest - 4096, 0)
6         self.seen = {c for c in self.seen if c > floor}
```

Rust v2 • replay.rs

```
1 pub fn commit(&mut self, ctr: u64) {
2     self.seen.insert(ctr);
3     if ctr > self.highest {
4         self.highest = ctr;
5         let floor = self.highest.saturating_sub(REPLAY_WINDOW);
6         self.seen = self.seen.split_off(&(floor + 1));
7     }
8 }
```




补上防重放的测试

channel.rs

crates/lanshare-proto/tests/

第 48-67 行 · 共 67 行

```
48 }
49
50 #[test]
51 fn replay_window_check_does_not_reserve() {
52     let mut w = ReplayWindow::new();
53     assert!(w.check(7));
54     assert!(w.check(7), "只检查不登记：解密失败的请求不能占位");
55     w.commit(7);
56     assert!(!w.check(7));
57 }
58
59 #[test]
60 fn replay_window_memory_stays_bounded() {
61     let mut w = ReplayWindow::new();
62     for ctr in 1..=100_000u64 {
63         assert!(w.check(ctr));
64         w.commit(ctr);
65     }
66     assert!(w.tracked() <= 4096, "只记住窗口内的计数器，实际 {}", w.tracked());
67 }
```



对方给的文件名，能直接用吗？

- `../../etc/passwd`: 带路径，想跳出共享文件夹
- `a<b>c:d.txt`: Windows 文件名里不允许的字符
- `CON`、`nul.txt`、`com1.tar.gz`: Windows 设备名
- `a + U+202E + gpj.exe`: 资源管理器里显示成 `aexe.jpg`
- 太长的名字: 按 UTF-8 不超过 200 字节，尽量保留扩展名



filename.rs: 三种坏字符

filename.rs

crates/lanshare-proto/src/

第 17-36 行 · 共 36 行

```

17         | '\u{2060}'..='\u{2064}'
18         | '\u{2066}'..='\u{2069}'
19         | '\u{FFF9}'..='\u{FFFB}'
20         | '\u{E0000}'..='\u{E007F}')
21     }
22
23     /// Windows 设备名：不管带不带扩展名都会被系统当成设备。
24     fn is_reserved(stem: &str) -> bool {
25         let upper = stem.trim().to_uppercase();
26         matches!(upper.as_str(), "CON" | "PRN" | "AUX" | "NUL" | "CONIN$" | "CONOUT$")
27         || ["COM", "LPT"].iter().any(|prefix| {
28             upper.strip_prefix(prefix).is_some_and(|rest| {
29                 let mut chars = rest.chars();
30                 matches!(
31                     (chars.next(), chars.next()),
32                     (Some('0'..='9' | '\u{00B9}' | '\u{00B2}' | '\u{00B3}'), None)
33                 )
34             })
35         })
36     }

```



按字节截断，不切坏汉字

filename.rs

crates/lanshare-proto/src/

第 29–48 行 · 共 48 行

```

29         let mut chars = rest.chars();
30         matches!(
31             (chars.next(), chars.next()),
32             (Some('0'..'9' | '\u{00B9}' | '\u{00B2}' | '\u{00B3}'), None)
33         )
34     })
35 })
36 }
37
38 /// 在不切断多字节字符的前提下，把字符串截到最多 `max` 字节。
39 fn truncate_bytes(s: &str, max: usize) -> &str {
40     if s.len() <= max {
41         return s;
42     }
43     let mut end = max;
44     while !s.is_char_boundary(end) {
45         end -= 1;
46     }
47     &s[..end]
48 }

```



sanitize_filename: 主流程

filename.rs

crates/lanshare-proto/src/

第 66–85 行 · 共 85 行

```

66     } else {
67         trimmed.to_string()
68     };
69     if named.len() <= MAX_NAME_BYTES {
70         return named;
71     }
72     // 太长：保留扩展名截前半部分；扩展名本身离谱地长就当成名字的一部分一起截。
73     let (base, ext) = match named.rfind('.') {
74         Some(i) if i > 0 && named.len() - i <= MAX_NAME_BYTES / 4 => named.split_at(i),
75         _ => (named.as_str(), ""),
76     };
77     let base = truncate_bytes(base, MAX_NAME_BYTES - ext.len());
78     let result = format!("{base}{ext}");
79     let result = result.trim_end_matches(['.', ' ']);
80     if result.is_empty() {
81         "unnamed".to_string()
82     } else {
83         result.to_string()
84     }
85 }

```




文件名清理：v1 和 v2

Python v1 • storage.py

```
1 def sanitize_filename(name):
2     name = name.replace("\\", "/").split("/")[-1]
3     name = _ILLEGAL_CHARS.sub("", name)
4     name = name.strip().strip(". ")
5     if not name:
6         return "unnamed"
```

Rust v2 • filename.rs

```
1 let last = name.rsplit(['/', '\\']).next().unwrap_or("");
2 let cleaned: String = last
3     .chars()
4     .filter(|&c| !is_illegal(c) && !is_invisible_format(c))
5     .collect();
6 let trimmed = cleaned.trim().trim_matches(|c| c == '.' || c == ' ');
7 if trimmed.is_empty() {
8     return "unnamed".to_string();
9 }
```



文件名测试：拷贝过来，挑几段讲

filename.rs

crates/lanshare-proto/tests/

从仓库拷贝

第 49-68 行 · 共 73 行

```

49
50 #[test]
51 fn removes_bidi_and_invisible_format_characters() {
52     // "a\u{202E}gpj.exe" 在资源管理器里显示成 "aexe.jpg", 用来把程序伪装成图片
53     assert_eq!(s("a\u{202E}gpj.exe"), "agpj.exe");
54     for c in ['\u{202A}', '\u{202B}', '\u{202C}', '\u{202D}', '\u{2066}', '\u{2067}', '\u{2068}', '\u{2069}',
55             '\u{200E}', '\u{200F}', '\u{200B}', '\u{FEFF}'] {
56         assert_eq!(s(&format!("re{c}port.pdf")), "report.pdf", "U+{:04X}", c as u32);
57     }
58 }
59
60 #[test]
61 fn truncates_long_names_keeping_extension() {
62     let result = s(&format!("{}", "中".repeat(150)));
63     assert!(result.ends_with(".mp4"));
64     assert!(result.starts_with('中'));
65     assert!(result.len() <= 200, "按 UTF-8 字节不超过 200, 实际 {}", result.len());
66 }
67
68 #[test]

```



踩坑：编译器拒绝“特洛伊源码”

```
Windows PowerShell

PS D:\code\lanshare> cargo test -p lanshare-proto --test filename
   Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
error: unicode codepoint changing visible direction of text present in comment
--> crates\lanshare-proto\tests\filename.rs:52:5
   |
52 | // "a💎gpj.exe" 在资源管理器里显示成 "aexe.jpg", 用来把程序伪装成图片
   | ^^^^^_^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   | |
   | | '\u{202e}'
   | this comment contains an invisible unicode text flow control codepoint
   |
= note: these kind of unicode codepoints change the way text flows on applications that support them, but can cause
confusion because they change the order of characters on the screen
= note: `[deny(text_direction_codepoint_in_comment)]` on by default
= help: if their presence wasn't intentional, you can remove them

error: could not compile `lanshare-proto` (test "filename") due to 1 previous error
```




lib.rs: 最后的模块和常量

lib.rs

crates/lanshare-proto/src/

第 23-42 行 · 共 48 行

```

23  /// 分块上传/下载的块大小。
24  pub const CHUNK: usize = 1 << 20;
25  /// 一次 RPC 请求明文的上限。
26  pub const MAX_RPC: usize = 64 * 1024;
27  /// ChaCha20-Poly1305 认证标签长度：密文 = 明文 + 16 字节。
28  pub const TAG_LEN: usize = 16;
29
30  /// 协议层面的错误。刻意不区分“密钥错”“被篡改”“计数器错”：对攻击者一律只说解不开。
31  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
32  pub enum ProtoError {
33      /// AEAD 认证失败：密钥、计数器、aad 或密文任意一处不对。
34      Decrypt,
35      /// PAKE 对端消息格式不对。
36      Pake,
37  }
38
39  impl std::fmt::Display for ProtoError {
40      fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
41          match self {
42              ProtoError::Decrypt => write!(f, "解密失败"),

```



测速程序

● ● ●

seal_speed.rs

crates/lanshare-proto/examples/

第 16–35 行 • 共 35 行

```
16      // 先热身一轮, 让 CPU 升频、缓存就位
17      let warm = seal(&key, 0, &aad, &plain);
18      assert_eq!(open(&key, 0, &aad, &warm).unwrap(), plain);
19
20      let started = Instant::now();
21      let mut sealed = Vec::with_capacity(TOTAL_MIB);
22      for ctr in 0..TOTAL_MIB as u64 {
23          sealed.push(seal(&key, ctr, &aad, &plain));
24      }
25      let seal_secs = started.elapsed().as_secs_f64();
26
27      let started = Instant::now();
28      for (ctr, block) in sealed.iter().enumerate() {
29          let back = open(&key, ctr as u64, &aad, block).expect("解密失败");
30          assert_eq!(back.len(), CHUNK);
31      }
32      let open_secs = started.elapsed().as_secs_f64();
33
34      println!("seal {:.0} MiB/s, open {:.0} MiB/s ({TOTAL_MIB} MiB, 1 MiB chunks)", TOTAL_MIB as f64 / seal_secs, TOTAL_MIB as f64 / open_secs);
35  }
```



全部测试，然后测速

```
Windows PowerShell

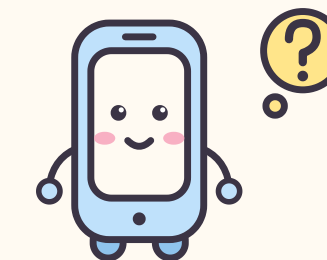
PS D:\code\lanshare> cargo test -p lanshare-proto
   Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
   Finished `test` profile [unoptimized + debuginfo] target(s) in 2.35s
..... (省略 6 行: src 里的单元测试, 0 个)
   Running tests\channel.rs (target\debug\deps\channel-ffe1e4ff30bebe6d.exe)

running 5 tests
test replay_window_rejects_duplicates_and_too_old ... ok
test any_mismatch_fails_to_open ... ok
test replay_window_check_does_not_reserve ... ok
test roundtrip_including_empty_and_large ... ok
test replay_window_memory_stays_bounded ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.03s

   Running tests\filename.rs (target\debug\deps\filename-1b867c0afaa9bb2a.exe)

running 9 tests
test absurd_extension_is_treated_as_part_of_name ... ok
test keeps_normal_names ... ok
test drops_directories ... ok
```



选择题

最高计数器是 10000，窗口 4096。计数器 6000 第一次到达，会怎样？

A 拒绝，因为它比最高的小

B 接受，因为它在窗口里，而且没用过

C 拒绝，因为太旧



选择题

最高计数器是 10000，窗口 4096。计数器 6000 第一次到达，会怎样？

A 拒绝，因为它比最高的小

窗口允许乱序，比最高值小不是拒绝的理由。

C 拒绝，因为太旧

只有小于等于 5904 的才算太旧。

B 接受，因为它在窗口里，而且没用过

6000 大于 10000 减 4096，也就是 5904，在窗口里，又没用过，所以接受。



选择题

为什么 **check** 只检查、不登记，要等解密成功后才 **commit**？

A 为了让代码跑得更快

B 防止伪造的请求把合法的计数器占掉

C 因为 BTreeSet 不能同时读和写



选择题

为什么 check 只检查、不登记，要等解密成功后才 commit?

A 为了让代码跑得更快

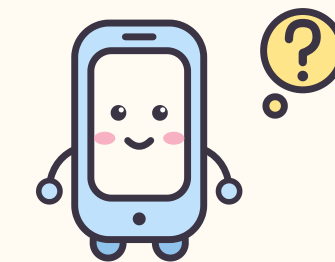
拆成两步和速度无关。

C 因为 BTreeSet 不能同时读和写

和集合的实现无关，这是协议层面的设计。

B 防止伪造的请求把合法的计数器占掉

捣蛋鬼没有密钥，伪造的密文一定解密失败。只有解密成功才登记，他就占不了位。



选择题

字符串“局域网”在 Rust 里，len 返回多少？

A 3

B 9

C 6



选择题

字符串“局域网”在 Rust 里，len 返回多少？

A 3

3 是字符数，Python 的 len 数的是字符。

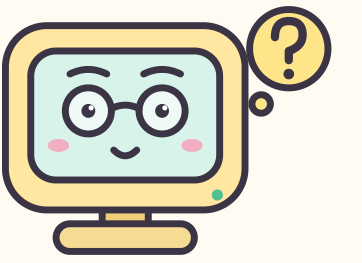
C 6

UTF-8 里常用汉字占 3 个字节，不是 2 个。

B 9

Rust 的 len 数的是 UTF-8 字节，一个汉字占 3 个字节，一共 9 个。

想一想



测试文件里那些看不见的字符，为什么一定要写成反斜杠 u 的转义，而不是直接粘贴字符本身？



想一想

测试文件里那些看不见的字符，为什么一定要写成反斜杠 u 的转义，而不是直接粘贴字符本身？

一是编译器根本不让：Rust 发现源码里有改变显示方向的字符，直接报错，防的就是特洛伊源码攻击。二是就算能编译，也没法审查：这些字符看不见，审代码的人根本不知道那里藏着什么。写成转义，每个字符都明明白白地摆在那里。这和我们清理文件名是同一个道理：看不见的东西，最危险。



协议核心完成了

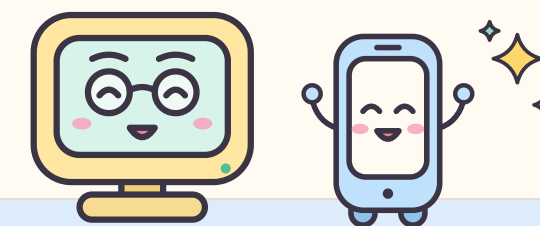
```
lanshare/  
├── crates/  
│   ├── lanshare/ 3 个文件  
│   └── lanshare-proto/  
│       ├── examples/  
│       │   └── seal_speed.rs 这一集写的  
│       └── src/  
│           ├── channel.rs  
│           ├── filename.rs 这一集写的  
│           ├── handshake.rs  
│           ├── lib.rs 这一集写的  
│           └── pake.rs
```

```
├── replay.rs 这一集写的  
├── tests/  
│   ├── channel.rs 这一集写的  
│   ├── filename.rs 这一集写的  
│   ├── pake.rs  
│   ├── vectors.json 从仓库拷贝  
│   └── vectors.rs 从仓库拷贝  
├── Cargo.toml  
├── lanshare-wasm/ 2 个文件  
├── web-tests/ 1 个文件  
└── Cargo.toml
```



这一集写了什么

- ✓ 防重放窗口：记住最高计数器和窗口内用过的，允许乱序，拒绝重复和太旧的
- ✓ check 只看不记，解密成功后才 commit；u64 做减法前先防溢出
- ✓ 文件名清理：最后一段路径、非法字符、看不见的格式字符、设备名、200 字节
- ✓ String 是自己拥有的字符串，按字节截断要找字符边界
- ✓ lanshare-proto 完成：27 个测试，测速程序



下一集

全课完结，感谢收看